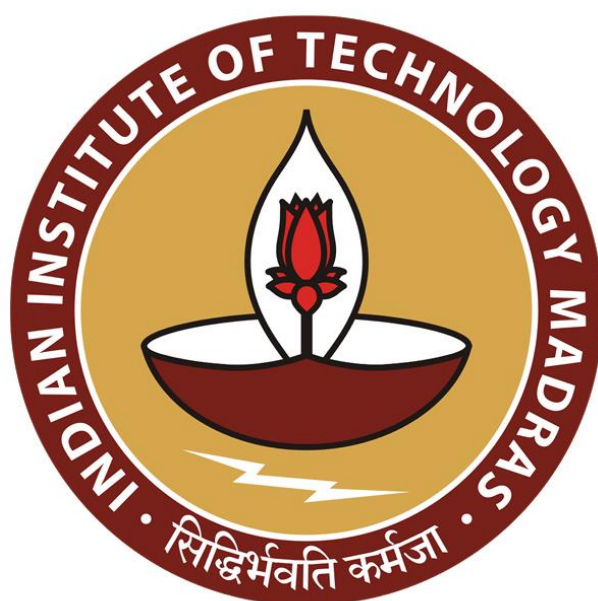




IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Subclasses and Inheritance

So, now that we have seen the philosophy behind object oriented programming and we have already seen how java classes are basically constructed. Let us move on to the first interesting aspect of java classes or object oriented programming in general which is the idea of sub classes and inheritance.

(Refer Slide Time: 00:33)

A Java class

- An **Employee** class
- Two private instance variables
- Some constructors to set up the object
- Accessor and mutator methods to set instance variables
- A public method to compute bonus

```
public class Employee{
    private String name;
    private double salary;

    // Some Constructors ...

    // "mutator" methods
    public boolean setName(String s){ ... }
    public boolean setSalary(double x){ ... }

    // "accessor" methods
    public String getName(){ ... }
    public double getSalary(){ ... }

    // other methods
    public double bonus(float percent){
        return (percent/100.0)*salary;
    }
}
```

Handwritten note: this.salary

So, let us begin with a standard class. So, suppose we want to keep track of people in an organization. So, we want to keep track of personnel in an organization. So, we create a class for instance called employee which will store information about each employee. So, each object will denote one person in the organization. So, in this employee class as usual we will have some private variables.

So, here we have two private instance variables one is name which is a string and the other is the salary which we store as a double. Then as we saw last week there will be some functions which are called implicitly when we create these objects. These are the constructors which will set up an employee with some values for the name and the salary

and then in order to update and access these private instance variables typically we will have functions which allow us to change the name and change the salary.

So, set name set salary and these accessor methods which will allow us to check the name and the salary by getting the value. So, we will have get name and get started and now we have one extra method which we will use later on. So, this is a function which basically computes the bonus due to an employee. So, the bonus depends on the salary. So, it returns.

So, if you tell the method how much percent bonus is supposed to be given this will compute the bonus as a percentage of that particular employee salary and since the salary is a local instance variable of each employee the value returned by bonus will be different depending on the salary of the employee even though the percentage that it is called with may be the same.

So, one thing to emphasize is that there is this reference to salary here okay and it is implicitly the salary of the object within which it is invoked. So, this is we can be made explicit by saying `this.salary` right we said that in earlier. So, the word `this` refers to the current object in which the context is set. So, you might remember that in python when we wrote these methods we used to have an extra first parameter which denoted the object on which the function is supposed to be there.

So, that we have a name for the current object which is always part of the function call in java this is implicit. So, we do not have an extra parameter but it is always implicit that there is a `this` associated with every method which is defined inside a class. So, just keep this in mind. So, when we say salary and there is no reference to salary inside this function it means that it must be an instance variable of this class.

(Refer Slide Time: 03:20)

- Managers are special types of employees with extra features

```
public class Manager extends Employee{
    private String secretary;
    public boolean setSecretary(name s){ ... }
    public String getSecretary(){ ... }
}
```

- Manager objects inherit other fields and methods from Employee
 - Every Manager has a name, salary and methods to access and manipulate these.
- Manager is a subclass of Employee
 - Think of subset



Madhavan Mukund

Subclasses and inheritance

Programming Concepts using Java 3/6

So, now let us look at sub classes. So, we have employees right and among the employees in the organization there is a hierarchy. So, there will be some employees who are managers. So, we want to record this information but managers are slightly different from employees because they might have other features which employees do not have but we do not want to rewrite the whole thing.

So, this is the idea of a sub class. So, we would like to say that a manager is like an employee but has more features and this is given in this definition. So, we say that the class Manager extends Employee. So, this is how java does this in python you use a bracket to show this subclassing. So, this means that manager is like an employee and it has more and what you do is you only define that part in manager which is different or which is more than employee.

So, you have everything that an employee has and in this case you additionally have another private instance variable called secretary which is again a string. So, this is the name of the. So, every manager in this organization presumably has a personal secretary who helps that manager with their work. So, we have a secretary and then corresponding to this secretary we have two new accessor and mutator methods to set the name of the secretary and to get the name of the secretary right.

So, this is at the starting point how you would define a sub class you give the new class name and you say that it extends. So, this is called the parent class right. So, you extend the parent class. So, implicitly this manager inherits the rest of its structure from the

employee right it inherits fields it inherits methods. So, it will inherit the earlier thing which was the name and the salary and the methods to set the name get the name set the salary get the salary also that method to compute the bonus right.

So, we say technically that manager is a sub class of employee now the way you should think of sub class is that you have employees as a whole set right and in that you have a smaller set which is manager right. So, it is just like subset. So, it is a special category its like you have all the numbers and then you have the even numbers. So, the even number is a subset of all the numbers it is all the numbers which are also divisible by 2.

So, there is an extra property. So, managers are employees but they have an extra property they also have a secretary for example.

(Refer Slide Time: 05:57)

Subclasses

- **Manager** objects do not automatically have access to private data of parent class.
 - Common to extend a parent class written by someone else
- How can a constructor for **Manager** set instance variables that are private to **Employee**?
- Some constructors for **Employee**
- Use parent class's constructor using **super**
- A constructor for **Manager**

```
public class Employee{
    ...
    public Employee(String n, double s){
        name = n; salary = s;
    }
    public Employee(String n){
        this(n,500.00);
    }
}

public class Manager extends Employee{
    ...
    public Manager(String n, double s, String sn){
        super(n,s); /* super calls Employee constructor */
        secretary = sn;
    }
}
```

Now one of the interesting points that differentiates this subclass thing from what we saw for instance in python is that this privacy is actually maintained across subclasses right. So, you have this employee class above in some sense right and below it you have this manager. So, part of what the manager needs is defined here and part of it is defined locally in the manager definition.

But the point is that what is defined here the private fields of employee are not accessible to the manager the manager cannot set the name or set the salary of the employee directly it can only use the set salary and get salary kind of thing okay. So, one

of the things that you might want is to create these things exactly like you would in an employee at construction time right.

So, you would like a constructor for manager to be able to set these values now it. So, happens that this employee has this thing but there may be other situations where you have private instance variables for which there is no direct way to set the value from outside but yet it is logical that since every manager is an employee even though it is private and hidden from the manager there must be a way for a manager to set this.

So, let us look at this situation. So, this is the employee and earlier we said there were some constructors. So, let us assume that these are two constructors which are available for employee right. So, employee has remember two private instance variables name and salary. So, the first one takes both arguments and sets them the second one takes only the name and puts a default salary in this case 500.

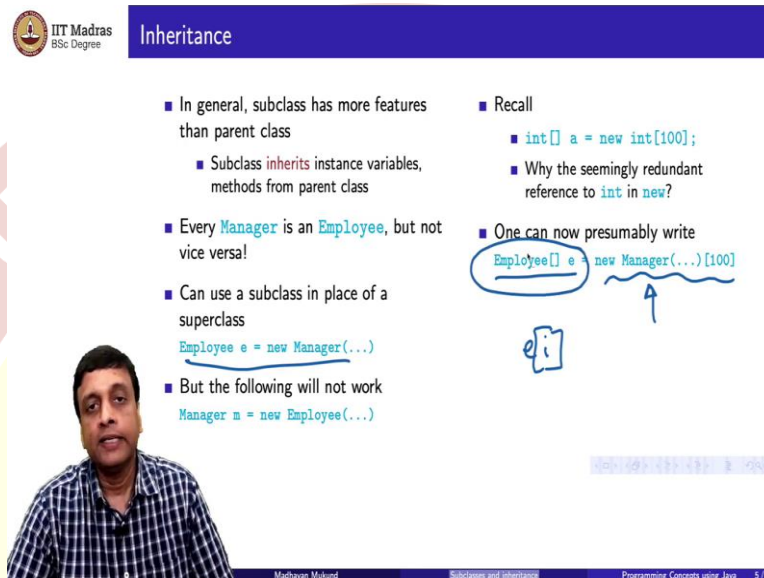
So, now we want to be able to set the name and the salary for a manager just like as we do here. So, the way we do that is by allowing the manager constructor to call the employee constructor at construction type and this is done through this word super which will be used in other situations also to refer to a function which is defined in the parent class in particular in a constructor just the word super is enough to say call the constructor from the parent class.

So, let us look at an example. So, supposing I have a manager constructor which takes three parameters name salary and the secretary name. Now of these only secretary name can be set by the manager because that is the only private instance variable that the manager has access to. So, how do you set the other ones well you invoke this constructor of employee right.

So, you say super(n,s) and this implicitly calls this constructor and therefore through that the name and the salary get it right. So, this is the trick. So, you can use super inside the child or the sub class constructor and it will call the parent construct. So, notice that we said normally that constructors you cannot call just like that right you can only call a constructor when you create an object now it is not that the manager is creating separately an employee object right.

So, you cannot call the constructor by saying give me a new employee because that is already part of giving a new manager. So, what you are saying is that the manager constructor has an extra privilege of being able to call the parent constructor by using the word super.

(Refer Slide Time: 09:10)



IIT Madras
BSc Degree

Inheritance

- In general, subclass has more features than parent class
 - Subclass *inherits* instance variables, methods from parent class
- Every **Manager** is an **Employee**, but not vice versa!
- Can use a subclass in place of a superclass
`Employee e = new Manager(...)`
- But the following will not work
`Manager m = new Employee(...)`
- Recall
 - `int[] a = new int[100];`
 - Why the seemingly redundant reference to `int` in `new`?
 - One can now presumably write
`Employee[] e = new Manager(...)[100]`

Handwritten annotations: A blue circle around `Employee[]` in the last bullet point, and a blue arrow pointing from the `new` keyword to the `int` in the code `int[] a = new int[100];`.

Madhavan Mukund Subclasses and inheritance Programming Concepts using Java 5/6

So, in general a sub class has more features as we have just seen with the manager than the parent class. So, it inherits the instance variables and the methods from parent class and it can define some more like it has defined in this case a secretary name is a new private instance variable and this gate secretary said secretary are new methods to manipulate this new instance variable.

So, in that subset setting every manager is an employee but not vice versa every even number is a number but not every number is an even number right. So, we remember the picture. So, this means that we can take a variable which is declared to be of type employee and where we expect an employee we can actually create an object of type manager the reason this works is that everything that an employee is capable of doing a manager is also capable of doing.

So, any method that you can invoke on an employee will also work for a manager. So, in that sense a manager has all the functionality of an employee and more right. So, this is what we will come back to this again but we mentioned before this is called a sub type

right. So, a sub class is a structural thing saying this class extends that class. So, that is a syntactic object but from a more mathematical point of view you are saying.

If you think about a data type which has certain allowed values and certain manipulations allowed on it a sub type has all the manipulations and data of the parent type and more. So, the manager is a sub type of employee and therefore in any context where you expect a manage employee you can also substitute a manager but remember that it is not true the other way right.

So, we cannot take an employee and put it in place of a manager. So, if we try this this will not work because if I create an employee object it has no notion of a secretary name for example. So, now if I take this employee which is assigned to m now the type of m as far as the declaration is, is a manager. So, every method that is allowed for manager should allow should work for m but if I have called m an employee obviously I cannot say m dot set secretary because it has no instance variable for secretary.

So, this is a one-sided thing right you can use a sub type or a sub class in place of the parent class but not vice versa. So, let us go back to something that we saw in the context of basic data types in java we saw this notion of an array we said that you will declare an array like this. So, you will give the parent type and the name of the array. So, you will say in and so this says a is an array of hint and then you will actually create an array by using a new thing.

So, this combines is an initialized declaration here right. So, you might ask if I already know that 'a' is an array of type int why do I again have to say int here right it seems to be a redundant thing. Because I have already declared 'a' to be an array of type int. So, when I say I want a new array I should be allowed to just specify the size I should say give me a new array of size 100 and it should infer that because 'a' is declared to be of type int that this is a new array of type int.

So, now that we have seen this notion of sub classes it becomes more interesting because we could have an array of not just in but of any user defined type. So, we could presumably have an array of employees. So, we could declare 'e' to be an array of employees but now because everywhere where I expect. So, everywhere I would expect

e[i] to be an employee but every employee could already be a manager. So, I can actually initialize an array of employees to an array of managers.

So, therefore the type that I declare at run time for the array needs to be compatible with the type that I have announced in advance in the declaration but it need not be exactly the same it can be a sub type. So, this is the reason why you have this seemingly redundant thing here which repeats the type in the instantiation.

(Refer Slide Time: 13:12)

IIT Madras
BSc Degree

Summary

- A subclass extends a parent class
- Subclass inherits instance variables and methods from the parent class
- Subclass can add more instance variables and methods
 - Can also override methods — later
- Subclasses cannot see private components of parent class
- Use `super` to access constructor of parent class

Madhavan Mukund Subclasses and inheritance Programming Concepts using Java 6 / 6

So, to summarize a sub class extends a parent class in java is literally the word extends is what is used the subclass inherits everything that the parent class has. It has all the instance variables and all the methods the parents class has it can add more instance variables and more methods. So, we saw this manager had a secretary name and get and set secretary name you can also change the way a method works right.

So, we saw this bonus ok. So, we said that the employee has a bonus defined now perhaps you want to say that a manager's bonus is computed definitely from employees bonus. So, you can also do that we will see that soon not in this particular lecture but the next one we will see that the sub class can also override methods. This is not obliged to just inherit the methods it can make the methods perform differently but one important thing is that the subclass cannot see the private components the parent class.

So, we saw last time we mentioned that an object of one type can see the private values of another object of the same type ok this is quite important because otherwise you will

not be able to sometimes say for example that copy constructor we cannot do right we cannot copy an object unless we can look up the instance variables of one and copy it into another one another idea we will look at is equality when are two objects the same if all their instance values are the same.

So, to compare two objects one of the objects will have to look at the instance variables of the other object. So, within a class there is no privacy right every object of one class can see all the private values of another object of the same class if it is passed to it. However a child class or a sub class does not have that same privilege and the logic behind this is that typically there are many classes which are given to you as part of the definition of the language or they have been defined earlier.

So, you might be working in a system in an environment where some classes are already defined by earlier programmers. So, it would be a mistake for the person using the sub class to go back and rewrite things in case the parent class changes right. So, you do not want the sub class to depend on the private parts of the parent class where the parent class is not under the control of the person who writes a subclass.

So, the whole idea is the separation of concerns right. So, the person who writes a piece of code should not be accidentally kind of forced to change their code because something that they are calling changes its representation as long as the public interface is the same everything should work. So, from that point of view a sub class may be written by a different developer from the parent class.

So, it is logical for the sub class to not have visibility of the parent class's private variables but we can get around that at least for constructors by using the super. So, we saw that when you construct the child class or the sub class you can call the parent class's super constructor at the time of calling this constructor. So, that you can set the private instance variables directly without having to worry about it